R26-CS-003 · COMPONENT 4

Browser Artifact Forensic Correlation Engine

Everything you need to explain, defend, and demonstrate your component to a strict panel — grounded in your actual code, not generic theory.

Owner S.A.O.D. Sandeepa · IT22252104

Stack Python · FastAPI · SQLite · LevelDB

Status 8/8 unit tests · 7/7 live detectors

01 The 30-Second Pitch

Say this first, before any question is asked. It frames everything else you say.

> "When an attacker compromises a browser, the evidence doesn't stay in one place — it scatters across history, cookies, saved passwords, downloads, and local storage, each in its own database. Existing forensic tools like Hindsight read these **one at a time**, so they never see the relationship between them. My component **normalizes six artifact types into one timeline** and runs **seven correlation algorithms** over a two-minute sliding window to find patterns that only exist **across** artifacts — for example, a credential stolen from one site reappearing as a cookie on another, followed by a download. Every finding is mapped to a **MITRE ATT&CK** technique and exported as a report a real SOC analyst could act on."

If they ask nothing else, that paragraph is your whole research contribution in one breath. Everything below is you being ready to go deeper on any single word in it.

02 Mental Model — Explain It Like a Detective Story

Strict panels respect an analogy that maps *exactly* onto the mechanism — not a loose metaphor. Use this one; it maps 1:1 onto your code.

A single-artifact tool is a detective who only reads one witness statement — "the history says he visited evil.com." That's a fact, not a case.

My engine is a detective who cross-references every witness statement inside the same 2-minute window — history says he visited evil.com, cookies say a session token appeared for evil.com 25 seconds later, and the credential store says a password was saved for evil.com 70 seconds after that. No single record proves an attack. **The correlation across three independent records does.**

That's the entire research contribution restated: **correlation is the detector, not any one artifact.**

03 The 5-Stage Pipeline

If asked "walk me through your system architecture," answer with this exact sequence — it is literally the order functions execute in `service.py`.

1. **Stage 1 — Extraction**

   `extractor.py`. Copies locked SQLite files safely (falls back to SQLite's online backup API if Chrome has the file locked), reads History/Cookies/Login Data/Downloads/Extensions, scans the Local Storage LevelDB store, and normalizes every artifact into one common event schema.

2. **Stage 2 — Single-artifact rules**

   `rules.py`. Six independent rules (R01–R06) flag individually suspicious events — a suspicious domain, a dangerous file extension, a sensitive cookie name — *before* any cross-referencing happens.

3. **Stage 3 — Cross-artifact correlation**

   `correlation.py`. The core contribution: seven detectors run over a 120-second sliding window, finding relationships *between* artifact types that Stage 2 cannot see on its own.

4. **Stage 4 — MITRE ATT&CK mapping**

   `mitre.py`. Every finding from Stage 2 and Stage 3 is assigned a technique ID, tactic, and Low/Medium/High severity.

5. **Stage 5 — Reporting**

   `reporter.py`. Produces a JSON report (archival), an HTML forensic report (human-readable), and a SIEM export (machine-ingestible by Splunk/QRadar/ELK).

04 The 7 Correlation Detectors

This is where a strict panel will spend the most time. Know each detector at three depths: **the one-line intuition**, **the actual mechanism**, and **a worked example**. All operate inside `WINDOW_SECONDS = 120`.

A / 7 Co-occurrence Analysis

`run_cooccurrence()`

Intuition: the more different artifact types touch the *same domain* inside one 2-minute window, the more suspicious that domain is.

MECHANISM

Groups every event in the window by domain. Counts distinct artifact types per domain. Score comes from a fixed table: `{1:0, 2:40, 3:70, 4:85, 5:100}`, then +5 per already-flagged event inside the cluster, capped at 100.

WORKED EXAMPLE

History visit to `evil.com` → cookie set for `.evil.com` 25s later → credential saved for `evil.com` 70s later. 3 artifact types → base score 70.

B / 7 Orphan Detection

`run_orphan_detection()`

Intuition: a cookie, credential, download, or local-storage entry that exists for a domain the user *never actually browsed to* was not created by normal user action — it was injected.

MECHANISM

Builds the set of every domain seen in *history*. Any cookie/credential/download/local-storage event whose domain has no matching history entry is flagged. Fixed severity weights: cookie 40, credential 60, download 70, extension 50, local storage 45.

WORKED EXAMPLE

A session cookie appears for `stealthy-inject.com` but the user's history has zero visits there → flagged as possible malware/script injection, not user action.

C / 7 Temporal Anomaly Detection

`run_temporal_anomaly()`

Intuition: not "2am is suspicious" for everyone — *this specific user's own* activity rhythm is learned, then deviations from it are flagged. This personalization is a deliberate design choice, and panels like hearing you defend it.

MECHANISM

Builds an hour-of-day histogram from the user's own history events. Any hour with ≤2% of total activity is "inactive." Any risk-type event (credential/download/cookie) landing in an inactive hour is flagged, weighted by artifact type (credential 70, download 50, cookie 40).

WORKED EXAMPLE

User's history shows dense browsing 9am–5pm, near-zero activity at 2am. A credential access at 2:05am is flagged — even though 2am isn't inherently suspicious for *every* user, it is for *this* one.

D / 7 Ordered Attack-Chain Detection

`run_attack_chain_detection()`

Intuition: stricter than co-occurrence — the artifact types must appear in a specific *order* that matches a known attack pattern, not just cluster together.

MECHANISM

Five predefined ordered sequences, each with a base score: `history→download→credential` (90), `history→credential→cookie` (85), `history→download→cookie` (80), `download→credential` (75), `credential→cookie` (70). A cursor scans events in timestamp order per domain and matches the exact sequence.

WORKED EXAMPLE

User browses `phish.net` → downloads `update.exe` 40s later → a credential for `phish.net` appears 90s after that. Matches the highest-value pattern — base score 90.

E / 7 Domain Risk Clustering

`run_domain_risk_clustering()`

Intuition: a rollup view — which single domain, across the *entire* extraction (not just one window), has accumulated the most concentrated risk?

MECHANISM

Per domain: `diversity_score = (types-1)×18`, `flagged_score = min(35, flagged_count×7)`, `density_score = min(20, event_count/3)`, plus a slice of the accumulated rule score. Only domains scoring ≥45 with ≥2 artifact types are reported.

WORKED EXAMPLE

`badactor.ru` touched by history + cookie + download + credential, with 4 of those events already rule-flagged → high diversity + high flagged score → reported as a concentrated-risk domain even if no single 2-minute window caught everything.

F / 7 Cross-Domain Credential Reuse

`run_credential_reuse()`

Intuition: the same saved username appearing on two or more *different* domains — one stolen password now unlocks multiple accounts. This is one of the two detectors added to complete the "7 detectors" commitment.

MECHANISM

Groups all credential events by normalized username, collects the distinct set of origin domains per user. ≥2 domains → flagged. `score = min(100, 45 + (domains-1)×20)`. Maps to MITRE **T1078 Valid Accounts**.

WORKED EXAMPLE

`alice@corp.com` saved on both `mail.corp.com` and `vpn.external-partner.net` → one leaked credential now has a wider blast radius than either domain alone would suggest.

G / 7 Download → Exfiltration Correlation

`run_download_exfil()`

Intuition: "drop the tool, then call home." A download followed within the window by outbound navigation to a *different* domain — the classic staging-then-exfil step. The second of the two added detectors.

MECHANISM

For every download, scans forward through history inside the 120s window for the first navigation whose domain differs from the download's source domain. Score 60 if the download itself was already risky, else 45. Maps to MITRE **T1567 Exfiltration Over Web Service**.

WORKED EXAMPLE

`collector.ps1` downloaded from `drop-zone.io` → 50 seconds later, navigation to `c2-beacon.xyz/upload` — a domain with no other relationship to the download.

05 DPAPI Credential Decryption

The panel will likely ask "how do you actually get the password out?" — this is the exact chain, step by step.

1. **Read the encrypted master key**

   Chrome stores an AES master key, itself encrypted, as base64 inside `Local State` (a JSON file next to the profile). Field: `os_crypt.encrypted_key`.

2. **Strip the DPAPI marker and unwrap the key**

   The first 5 bytes are the literal marker `b"DPAPI"`. After stripping, the remaining bytes are passed to Windows' own `CryptUnprotectData` function — called directly via Python's `ctypes` against `crypt32.dll`, so **no third-party library is trusted with the secret**. This returns the raw 32-byte AES-256 key.

3. **Decrypt each password blob**

   Every password in the `logins` table is stored as `b"v10"` or `b"v11"` + a 12-byte nonce + AES-256-GCM ciphertext. Using the recovered key, each blob is decrypted independently.

4. **Mask before writing to disk**

   By default `REVEAL_PLAINTEXT = False` — the report stores a masked preview (`h*****3`: first char, last char, length) instead of the live password. Decryption success is still provable via the `status` field and character count, without ever persisting a real secret to a report file.

Why this is defensible research, not just a feature

DPAPI keys are bound to the *currently logged-in Windows user* — the same user whose browser profile is being scanned. That is exactly the deployment model here (the forensic engine runs as the profile owner), so this is not a security bypass; it recovers a secret the OS already trusts this process to read. If asked "isn't this exactly what malware does?" — the honest answer is: yes, the mechanism is identical to what credential-stealing malware uses, which is precisely why understanding it is valuable forensic and defensive knowledge, and why the masking default exists.

06 MITRE ATT&CK Mapping

Every finding — from Stage 2 rules and all 7 Stage 3 detectors — is translated into a standardized technique so the output means something to a real security team, not just to this project.

PATTERN DETECTED	TECHNIQUE	TACTIC	SEVERITY
Cookie + credential co-occurrence	T1539	Credential Access	High
History + cookie + credential	T1185	Collection	High
Orphan download (no history)	T1105	Command & Control	High
Dangerous file download	T1204.002	Execution	Medium
Risky extension permissions	T1176	Persistence	Medium
Cross-domain credential reuse	T1078	Persistence	Medium
Download → outbound navigation	T1567	Exfiltration	Medium

Severity itself is computed, not hand-picked per finding: `score ≥ 70 → High`, `score ≥ 40 → Medium`, else `Low` (see `_severity()` in `mitre.py`).

07 Proving It Works — Three Kinds of Evidence

A strict panel doesn't want "it works, trust me." Give them three independent kinds of proof, escalating in rigor.

1 · Unit correctness — 8/8 scenarios pass

`test/C4/test_correlation.py` constructs a synthetic attack for *each* of the 7 detectors individually (plus a combined MITRE-mapping pass) and asserts the exact expected finding is produced. This proves each algorithm is **individually correct**, isolated from the others.

2 · Integration proof — 7/7 detectors fire together

`test/C4/demo_attack_profile.py` plants one coherent 192-event breach narrative (phishing → drive-by download → credential theft → exfil callback → orphan injection → credential reuse) and runs it through the *entire real pipeline*, not mocks. Result: all 7 detectors fire, 25 MITRE findings (13 High). This proves the full system works **end-to-end, together**, not just in isolated unit tests.

3 · Cryptographic proof — real DPAPI key recovery

The AES master key was recovered from a live Windows profile (32 bytes — correct for AES-256) and used to decrypt a value encrypted in Chrome's exact `v10` blob format, round-tripping back to the original plaintext. This proves the decryption chain works against **real Windows cryptographic APIs**, not a simulated stand-in.

EVIDENCE TYPE	WHAT IT PROVES	RESULT
Unit tests	Each detector is individually correct	`8 / 8 passed`
Demo pipeline	All detectors work together, end-to-end	`7 / 7 fired · 25 findings`
DPAPI round-trip	Decryption works against real Windows crypto	`32-byte key recovered · round-trip OK`

08 Live Demo Script (Run In Front Of The Panel)

Rehearse this exact sequence beforehand so it's muscle memory. Run everything from the project root.

1. **Open a terminal in the project root**

   `d:\All Years Assignments\Y4 S1\Final Research Project\R26-CS-003`

2. **Run the unit test suite**

   `python test/C4/test_correlation.py` — say out loud: "this proves each of my 7 correlation algorithms individually detects what it's designed to detect." Expect `TEST SUMMARY 8/8 passed`.

3. **Run the live attack demo**

   `python test/C4/demo_attack_profile.py` — say: "this plants a realistic multi-stage breach and runs it through the complete pipeline live." Expect `Detectors fired: 7/7` and the HTML report opens automatically.

4. **Walk the panel through the opened HTML report**

   Point at the Executive Summary, then the MITRE ATT&CK Findings table (name a technique ID and what it means), then the flagged-event timeline (show a specific event's `anomaly_reasons` explaining *why* it was flagged).

5. **(If time allows) show the artifact manifest**

   Point at the SHA-256 hash column — say: "this preserves chain of custody; the extracted evidence is hash-verifiable."

If a command fails on the day: the most likely cause is Chrome/Edge being open and locking the SQLite files. Close the browser and rerun. The extractor also has an automatic fallback (SQLite's online backup API) for exactly this case — mentioning that fallback out loud turns a live glitch into a demonstration of robustness.

09 Panel Q&A Bank

The hardest, most likely questions a stick panel will actually ask — with answers you can defend, not scripts to recite verbatim.

Q Why 120 seconds? Why not 30 seconds, or 10 minutes?

120 seconds was chosen as a practical window for *automated, machine-speed* multi-stage sequences — a script chaining credential theft, cookie set, and download typically completes in well under two minutes, while normal human browsing rarely touches 3+ unrelated artifact types on the same domain that fast. **Be honest:** this is a fixed constant informed by the attack scenarios modeled, not empirically tuned against a large labelled dataset — and say that's explicitly listed as future work (a labelled real-world corpus to tune it). Owning the limitation is stronger than pretending it's derived from a formula.

Q How is this different from just writing SQL joins across the databases?

Three reasons:

- The databases use *different* timestamp formats and schemas (Chrome epoch microseconds vs. Unix time vs. LevelDB records) — the extractor's job is normalizing all of them into one comparable event schema first.
- A raw join can't express "ordered sequence within a sliding window" (Detector D) or "personal baseline deviation" (Detector C) — those require stateful, per-user computation, not a static join.
- Every finding needs a computed risk score and a MITRE mapping, which is analysis logic, not retrieval.

Q What is your false-positive rate?

Be honest here: no formal false-positive rate has been measured against a large labelled real-world dataset — the evaluation performed is scenario-based (8 unit tests + 1 integrated demo), which proves the detectors work *correctly* against known attack patterns, not their precision/recall against real-world noisy data. Frame this directly as a limitation and future-work item: a labelled corpus of real benign + malicious browsing sessions would let false-positive/false-negative rates be measured empirically. Don't invent a number.

Q Isn't decrypting saved passwords exactly what malware (infostealers) does?

Yes — and that's the point. Understanding the DPAPI + AES-GCM extraction chain used by real credential-stealing malware is what lets a defender detect and respond to it. This is standard practice in digital forensics and incident response (the same reasoning behind tools like Mimikatz/pypykatz, which the project's own proposal names). The responsible-disclosure design choice is the **masking default** — decrypted values are proven recoverable (status + length) without persisting live plaintext secrets into report files that could themselves become a target.

Q Why only Windows / DPAPI? What about Mac or Linux users?

DPAPI is a Windows-specific OS API, and the project's browser profile (via the shared Playwright session) targets a Windows deployment. Cross-platform credential decryption would need a separate backend per OS — macOS Keychain, Linux gnome-keyring/kwallet — each with different access APIs. This is explicitly scoped as future work, not an oversight; the architecture (a dedicated `crypto.py` module called from the extractor) is already structured so a second backend could be added without touching the correlation or MITRE layers.

Q How do you know your risk-scoring weights (e.g. co-occurrence 40/70/85/100, orphan weights) are the "right" numbers?

They encode a reasonable prior ordering — more artifact types co-occurring is worse, credentials are riskier than cookies, downloads riskier still — validated by the fact that all 8 test scenarios and the demo correctly separate attack events from benign ones. They were *not* derived from a statistical model fit to labelled data. If pushed: "the relative ordering is defensible; the exact numeric gaps would benefit from calibration against a larger dataset, which is future work."

Q What's the actual novel contribution here versus existing tools like Hindsight?

Hindsight (and similar tools) extract and display artifacts *per type* — a history tab, a cookie tab. It does not correlate *across* types. The contribution here is specifically the **7 cross-table detectors** that find relationships invisible to any single-artifact view: an orphan artifact only makes sense relative to the history table; credential reuse only makes sense comparing multiple rows of the login table against each other; download-then-exfil only makes sense joining downloads against history by time. None of that exists if you only ever look at one table at a time.

Q **How does this integrate with the other 3 components?**

Architecturally, all four components share the FastAPI backend (`core/main.py`) and the same Playwright browser session, and each exposes its findings via REST endpoints (C4's are under `/forensic/*`) so a unified dashboard can display all four. Functionally, C4 is the *retrospective* layer — where C1–C3 detect threats in real time (malicious extensions, BiTB phishing, C2 beacons), C4 reconstructs *after the fact* what already happened across the browser's stored artifacts, which is valuable even if a real-time detector missed the initial compromise.

10 Winning the Room

Lead with the pitch, not the code

Open with the 30-second pitch from Section 1 before anyone asks a question. It frames every follow-up in your terms, not theirs.

Own your limitations before they find them

A panel trusts a presenter more, not less, for saying "this is a fixed constant, not empirically tuned — that's listed as future work." Hiding a gap and getting caught costs far more than naming it first.

Always answer with a mechanism

Never say "it detects suspicious activity." Say *what field*, *what threshold*, *what formula*. "Score is 45 base plus 20 per extra domain, capped at 100" is unassailable; "it gives it a risk score" invites another question.

Use the demo as your answer, not your intro

When a question is answerable by running the demo, say "let me show you" and run it live. A panel member who watches 7/7 detectors fire in front of them argues less than one who only heard you claim it.

Know one number cold per detector

Memorize just the score formula or threshold for each of the 7 — e.g. "orphan credential = weight 60," "reuse = 45 + 20 per extra domain." One precise number beats a paragraph of description.

Redirect "why not X" to "that's future work"

For anything genuinely out of scope (cross-platform DPAPI, ML-tuned weights, full LevelDB decoding) — don't defend it as done. Name it as a deliberately scoped boundary with a one-sentence reason.
